Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică

Laboratory Work 8:

Track B: The Same Calls Inside Your Own OS (xv6)

Elaborated: gr. FAF-243
Poiata Calin

Verified:
Patricia Reițman

Chișinău – 2026

Table of Contents

Environment & Toolchain 1
Purpose of the Laboratory Work 1
Part 1. `run`, Inside xv6 1
1. The Three Steps of the Shell in xv6 1
2. Implementation: `user/run.c` 1
3. Makefile Registration and Build 2
4. Execution Results: `echo` Target 2
5. Binary Substitution: Executing `ls` 3
6. Handling Non-Existent Executables 3
7. Answers to Part 1 Observe Questions 4
Part 2. `pingpong` 4
1. Inter-Process Synchronization Through Pipes 4
2. Implementation: `user/pingpong.c` 5
3. Execution Results in QEMU 5
4. Answers to Part 2 Observe Questions 6
Stretch Goal: `primes` (Concurrent Sieve of Eratosthenes) 6
1. Algorithmic Architecture 6
2. Implementation: `user/primes.c` 7
3. Resource Management & Compiler Directives 8
4. Execution Results in QEMU 8
Comparative Analysis: Linux vs xv6 Process Model & IPC 8
Conclusion 9

Environment & Toolchain

- **Operating System Kernel:** xv6-riscv (MIT teaching operating system, 64-bit RISC-V)
- **Host Machine:** Arch Linux x86_64, Linux Kernel 7.2.9-arch1-1
- **Emulator:** QEMU emulator version 10.2.1 (`qemu-system-riscv64`)
- **Cross-Compiler:** `riscv64-linux-gnu-gcc` (GCC 15.2.1)
- **Linker & Binary Utilities:** GNU binutils for RISC-V 64-bit (`riscv64-linux-gnu-ld` , `objdump`)
- **Document Processor:** Typst 0.15.1 (`typst`) and TeX Live 2026 (`pdflatex`)

Purpose of the Laboratory Work

1. Verify that fundamental Unix process management primitives (`fork` , `exec` , `wait`) function identically inside a minimalist monolithic kernel (xv6) running on bare-metal RISC-V hardware emulation as they do under GNU/Linux.
2. Implement a user-space command launcher (`user/run.c`) in xv6 that replicates the core process orchestration cycle of a shell.
3. Investigate binary substitution semantics and process exit status propagation within xv6 without the C standard library (`glibc`), relying purely on direct kernel system calls (`sys_fork` , `sys_exec` , `sys_wait` , `sys_exit`).
4. Implement inter-process communication using kernel pipes in `user/pingpong.c` , transferring data between parent and child processes and observing bidirectional synchronization guarantees.
5. Analyze pipe buffer directionality and blocking read semantics to understand why half-duplex pipes require two separate channel descriptors for bidirectional protocols.
6. Design and implement the concurrent sieve of Eratosthenes (`user/primes.c`) using an interconnected pipeline of processes and pipes, rigorously managing file descriptor limits (`NOFILE = 16`).

Part 1. `run` , Inside xv6

1. The Three Steps of the Shell in xv6

In Laboratory Work 7, we constructed `run.c` on GNU/Linux using POSIX standard library functions (`fork` , `execlp` , `waitpid`). In xv6, there is no `glibc` , no dynamically linked libraries, and no high-level POSIX wrappers. Every call directly invokes a kernel trap (`ecall`) into xv6′s system call dispatch table.

Despite the extreme architectural simplification, the three essential steps required to launch an external executable remain completely identical:

1. **Process Duplication (`fork`):** Allocates a new `struct proc` from the kernel table, allocates a new page table duplicating the parent's user memory mappings, copies the parent's file descriptor handles, and sets the child's state to `RUNNABLE` .
2. **Image Replacement (`exec`):** Reads the ELF executable headers from the xv6 file system, allocates fresh user page table pages, loads code and data segments, sets up the initial user stack with string arguments, and points the program counter (`trapframe→epc`) to the entry point.
3. **Synchronization and Reaping (`wait`):** Puts the parent process into `SLEEPING` state until any child terminates (`ZOMBIE`), whereupon the child's resources (memory pages, kernel stack) are reclaimed and its exit integer is returned.

2. Implementation: `user/run.c`

The implementation of `user/run.c` conforms to xv6 userland conventions:

```c
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();                     // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){                         // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                 // step 2: become echo
    printf("exec failed\n");            // runs only if exec failed
    exit(1);
  }
  int status;
  wait(&status);                        // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

3. Makefile Registration and Build

To integrate `run.c` into the xv6 filesystem image (`fs.img`), the user executable must be added to the `UPROGS` variable in the xv6 `Makefile` :

```
UPROGS=\
  $U/_cat\
  $U/_echo\
  $U/_forktest\
  $U/_grep\
  $U/_init\
  $U/_kill\
  $U/_ln\
  $U/_ls\
  $U/_mkdir\
  $U/_rm\
  $U/_sh\
  $U/_stressfs\
  $U/_usertests\
  $U/_grind\
  $U/_wc\
  $U/_zombie\
  $U/_mystat\
  $U/_find\
  $U/_run\
  $U/_pingpong\
  $U/_primes\
```

Building and starting xv6 under QEMU:

```
$ make qemu
...
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

4. Execution Results: `echo` Target

Executing `run` inside the xv6 shell:

```
$ run
parent: my pid is 3
```

```
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

Observations:

- The parent process PID was 3 (since PID 1 is `init` and PID 2 is the login `sh`).
- The child process was assigned PID 4.
- The string `"hello from the child"` was emitted.
- The parent successfully reaped child 4 with exit status `0`.

5. Binary Substitution: Executing `ls`

Modifying the argument array and `exec()` call in `user/run.c`:

```
char *args[] = { "ls", 0 };
exec("ls", args);                 // step 2: become ls
```

Rebuilding and running:

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35864
echo           2 4 34760
forktest       2 5 18672
grep           2 6 35240
init           2 7 34264
kill           2 8 34968
ln             2 9 34776
ls             2 10 37376
mkdir          2 11 35024
rm             2 12 35008
sh             2 13 46000
stressfs       2 14 36040
usertests      2 15 168672
grind          2 16 45472
wc             2 17 36320
zombie         2 18 34384
mystat         2 19 35912
find           2 20 37288
run            2 21 35688
pingpong       2 22 36240
primes         2 23 37120
console        3 24 0
parent: child 4 ended with status 0
```

The child process transformed completely into the `ls` utility, dumping the root directory entries, and returned status 0.

6. Handling Non-Existent Executables

Testing failure handling by substituting a non-existent binary path `nosuchprog`:

```
char *args[] = { "nosuchprog", 0 };
exec("nosuchprog", args);
printf("exec failed\n");
exit(1);
```

Execution output:

```
$ run
parent: my pid is 3
child: my pid is 4
```

```
exec failed
parent: child 4 ended with status 1
```

When `exec` fails to find `/nosuchprog` in the xv6 file system, `exec()` returns `-1` to the child process. The child proceeds to the next line, prints `"exec failed\n"`, and terminates with `exit(1)`. The parent's `wait(&status)` retrieves status `1`.

7. Answers to Part 1 Observe Questions

1. Write the three calls that `run.c` uses in xv6 and the three calls `run.c` used on Linux in Lab 7. What is the same?

- **Calls in xv6 (Lab 8):** `fork()`, `exec(path, argv)`, `wait(&status)`.
- **Calls in Linux (Lab 7):** `fork()`, `execlp(file, arg0, ...)` (or `execve`), `waitpid(pid, &status, options)`.
- **What is the same:** Both operating systems enforce the classic Unix process creation abstraction:
  1. Process cloning is fully isolated from program loading.
  2. The parent creates an exact duplicate via `fork()`.
  3. The child replaces its instruction stream, memory, and register context via an `exec` variant.
  4. The parent synchronizes execution and reclaims process resources via `wait` / `waitpid`.

2. Who prints "hello from the child": the code of `run`, or `echo`?

The code of `echo` prints it. When `exec("echo", args)` succeeds inside `sys_exec`, the kernel discards the original code, data, and stack segments belonging to `run.c`. The ELF binary of `echo` is loaded from disk into the child's address space. The child's program counter is initialized to the entry symbol of `echo`, which iterates through `argv` and emits `"hello from the child\n"` to file descriptor 1. The instructions in `run.c` following `exec` no longer exist in memory.

3. Why is the status 1 here and 127 on Linux? (Look at the exit line after exec in both programs.)

The exit status retrieved by `wait()` is simply the integer parameter passed to `exit()` by the terminating child process:

- In Linux Lab 7, `run.c` explicitly wrote `exit(127);` following standard POSIX shell conventions where exit code 127 signifies "command not found".
- In xv6 Lab 8, `user/run.c` explicitly wrote `exit(1);` upon detecting that `exec` failed.

The kernel simply passes this numeric argument from `sys_exit` to the parent in `sys_wait`.

Part 2. `pingpong`

1. Inter-Process Synchronization Through Pipes

In Part 2, we implement bidirectional token passing between two processes using kernel pipes. A pipe in Unix provides a bounded circular byte buffer managed by the kernel. Because pipes are strictly half-duplex (unidirectional), two distinct pipes are required for bidirectional communication:

- `p2c` (Parent-to-Child): The parent writes a single byte `'x'` into `p2c[1]`; the child blocks reading from `p2c[0]`.
- `c2p` (Child-to-Parent): Upon receipt, the child writes the byte into `c2p[1]`; the parent blocks reading from `c2p[0]`.

2. Implementation: `user/pingpong.c`

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b = 'x';

  if(pipe(p2c) < 0 || pipe(c2p) < 0){
    fprintf(2, "pipe failed\n");
    exit(1);
  }

  int pid = fork();
  if(pid < 0){
    fprintf(2, "fork failed\n");
    exit(1);
  }

  if(pid == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    if(read(p2c[0], &b, 1) != 1){
      fprintf(2, "child read failed\n");
      exit(1);
    }
    printf("%d: received ping\n", getpid());
    if(write(c2p[1], &b, 1) != 1){
      fprintf(2, "child write failed\n");
      exit(1);
    }
    close(p2c[0]);
    close(c2p[1]);
    exit(0);
  }

  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  if(write(p2c[1], &b, 1) != 1){
    fprintf(2, "parent write failed\n");
    exit(1);
  }
  if(read(c2p[0], &b, 1) != 1){
    fprintf(2, "parent read failed\n");
    exit(1);
  }
  printf("%d: received pong\n", getpid());
  close(p2c[1]);
  close(c2p[0]);
  wait(0);
  exit(0);
}
```

3. Execution Results in QEMU

After compiling and executing inside xv6:

```
$ pingpong
6: received ping
5: received pong
```

The execution flow unfolded deterministically:

1. Child (PID 6) blocked in `read(p2c[0], &b, 1)`.
2. Parent (PID 5) wrote `'x'` to `p2c[1]`, then blocked in `read(c2p[0], &b, 1)`.

3. Child woke up, printed `"6: received ping"`, wrote `'x'` to `c2p[1]`, closed its descriptors, and exited.
4. Parent woke up, printed `"5: received pong"`, reaped the child via `wait(0)`, and exited cleanly.

4. Answers to Part 2 Observe Questions

1. Why do you need TWO pipes, and not one for both directions?

Pipes in Unix and xv6 are strictly unidirectional (half-duplex). A single pipe represents a single shared FIFO circular buffer protected by a spinlock. If only one pipe `p[2]` were used for both communication directions:

1. When the parent writes the ping byte into `p[1]`, both the parent and child have open read handles on `p[0]`.
2. If the parent immediately invokes `read(p[0], &b, 1)`, a race condition ensues: the parent may read back the exact byte it just wrote before the child scheduler slice executes.
3. Similarly, if the child attempted to reply through the same pipe, it might immediately consume its own reply.

Two dedicated pipes (`p2c` for parent-to-child and `c2p` for child-to-parent) ensure separate, independent buffers, eliminating race conditions.

2. Nothing in your code says "wait until the child has replied". Which call makes the parent wait?

The system call **`read(c2p[0], &b, 1)`** makes the parent wait.

In xv6's `pipe.c`, the `piperead()` function acquires the pipe's lock and inspects the buffer counter (`nread` == `nwrite`). When the buffer is empty, it puts the calling process to sleep on the pipe channel:

```
while(pi->nread == pi->nwrite && pi->writeopen){
  sleep(&pi->nread, &pi->lock);
}
```

The parent process transitions from `RUNNING` to `SLEEPING` state in the kernel. It is only awakened when the child executes `write(c2p[1], &b, 1)` (which triggers `wakeup(&pi->nread)` in `pipewrite`). Thus, inter-process synchronization is achieved inherently via blocking I/O without active busy-waiting or explicit `sleep()` calls.

Stretch Goal: `primes` (Concurrent Sieve of Eratosthenes)

1. Algorithmic Architecture

The concurrent sieve of Eratosthenes (invented by M. D. McIlroy) models prime number generation as an expanding pipeline of concurrent processes connected by pipes:

```
[Main Generator 2..35] ---> [Filter 2] ---> [Filter 3] ---> [Filter 5] ...
```

1. The main process feeds numbers $2, 3, 4, ..., 35$ into a pipe.
2. The first stage process reads the first integer available from its left pipe: this number p is guaranteed to be prime (starting with 2). It prints `"prime p"`.

3. The stage process creates a new pipe `pright`, forks a child running the next sieve stage, and forwards through `pright[1]` only those remaining incoming integers n that are **not divisible** by p ($n \bmod p \neq 0$).
4. When its left pipe reaches EOF (when all writers have closed their descriptors), the stage closes its write descriptor to signal EOF to the next stage, and calls `wait(0)` to reap its downstream child.

2. Implementation: `user/primes.c`

```c
#include "kernel/types.h"
#include "user/user.h"

#pragma GCC diagnostic ignored "-Winfinite-recursion"

void
sieve(int pleft[2])
{
  int p;
  close(pleft[1]);
  if(read(pleft[0], &p, sizeof(p)) != sizeof(p)){
    close(pleft[0]);
    exit(0);
  }

  printf("prime %d\n", p);

  int pright[2];
  if(pipe(pright) < 0){
    fprintf(2, "pipe failed\n");
    close(pleft[0]);
    exit(1);
  }

  int pid = fork();
  if(pid < 0){
    fprintf(2, "fork failed\n");
    close(pleft[0]);
    close(pright[0]);
    close(pright[1]);
    exit(1);
  }

  if(pid == 0){
    close(pleft[0]);
    sieve(pright);
  } else {
    close(pright[0]);
    int n;
    while(read(pleft[0], &n, sizeof(n)) == sizeof(n)){
      if(n % p != 0){
        write(pright[1], &n, sizeof(n));
      }
    }
    close(pleft[0]);
    close(pright[1]);
    wait(0);
    exit(0);
  }
}

int
main(int argc, char *argv[])
{
  int p[2];
  if(pipe(p) < 0){
    fprintf(2, "pipe failed\n");
    exit(1);
  }

  int pid = fork();
  if(pid < 0){
    fprintf(2, "fork failed\n");
    exit(1);
  }
```

```
  if(pid == 0){
    sieve(p);
  } else {
    close(p[0]);
    for(int i = 2; i <= 35; i++){
      write(p[1], &i, sizeof(i));
    }
    close(p[1]);
    wait(0);
    exit(0);
  }
  exit(0);
}
```

3. Resource Management & Compiler Directives

Two critical engineering constraints had to be solved:

1. **GCC 15 `-Winfinite-recursion` Warning:** Because `sieve()` calls itself inside `if (pid == 0)`, GCC 15.2.1 infers potential infinite recursion across child process paths and fails under `-Werror`. Adding `#pragma GCC diagnostic ignored "-Winfinite-recursion"` prevents compiler stoppage while maintaining full recursive process instantiation.
2. **xv6 File Descriptor Limit (`NOFILE = 16`):** In xv6, each process can hold a maximum of 16 simultaneously open descriptors (`param.h`). Without meticulous closing of unused ends (`close(pleft[0])`, `close(pleft[1])`, `close(pright[0])`), file descriptors would leak across child processes, quickly exhausting the per-process table and causing `pipe()` to fail.

4. Execution Results in QEMU

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
```

All 11 prime numbers in the range $[2, 35]$ were calculated concurrently and emitted in strictly ascending order without deadlocks or leaked file descriptors.

Comparative Analysis: Linux vs xv6 Process Model & IPC

Dimension	Linux (Lab 7)	xv6-riscv (Lab 8)
Architecture	Monolithic general-purpose kernel (x86_64, preemptive SMP, glibc, dynamic linking).	Minimalist educational monolithic kernel (RISC-V 64-bit, static ELF loading, no glibc).
Process Duplication	`fork()` with Copy-on-Write (CoW) virtual memory pages; fast allocation.	`fork()` allocates fresh physical pages and copies memory bytes eagerly via `uvmcopy()`.
Program Execution	`execlp()` / `execve()` parses ELF binaries, links shared libraries, maps stack and heap.	`exec()` parses static ELF headers, maps fixed text/data/stack, and constructs argv array directly.

Dimension	Linux (Lab 7)	xv6-riscv (Lab 8)
Process Reaping	`waitpid(pid, &status, options)` offers non-blocking polling (`WNOHANG`) and specific PID targeting.	`wait(&status)` sleeps until **any** child exits, returning its PID and status; no PID targeting.
Pipe Buffering	Dynamic circular buffer (default 65,536 bytes); POSIX `O_NONBLOCK` support.	Fixed circular buffer (`PIPESIZE = 512` bytes in `pipe.c`); blocking read/write only.
Descriptor Limit	Default soft limit 1,024 descriptors per process (`RLIMIT_NOFILE`), configurable up to millions.	Strict constant `NOFILE = 16` descriptors per process; `NFILE = 100` system-wide.

Conclusion

Laboratory Work 8 demonstrated the elegance and universality of the Unix process model by reproducing core process orchestration and inter-process communication inside xv6:

1. **Universality of Unix Primitives:** Demonstrated that the triad of `fork()` , `exec()` , and `wait()` operates identically in a 3,000-line educational kernel running on RISC-V hardware emulation as it does in production Linux kernels.
2. **Decoupling Shell Logic:** Verified that external command execution is achieved through address space substitution without modifying or retaining parent execution state.
3. **Synchronization Through Blocking I/O:** Built a bidirectional `pingpong` communication exchange and confirmed that blocking pipe reads naturally synchronize parent and child processes without busy-waiting or explicit timer delays.
4. **Concurrent Process Pipelines:** Successfully implemented McIlroy's concurrent prime sieve (`primes.c`), verifying pipeline expansion, stream filtering, and strict file descriptor management under xv6′s 16-descriptor limitation.